

Datos del alumno

Nombre: Manuel Ramirez Galvan

Numero de actividad: 2

Fecha: 29/01/2025

✓ Actividad Pixeles Resolución y Profundidad

Objetivos: Cargar imágenes y convertirlas entre diferentes formatos. Manipular píxeles individuales y analizar colores. Explorar el efecto de cambiar la resolución y la profundidad de color.

✓ Ejercicio 1: Importar bibliotecas necesarias

```
import cv2
import matplotlib.pyplot as plt
```

✓ Ejercicio 2: Cargar y mostrar una imagen en escala de grises

Objetivo: Aprende a cargar una imagen en escala de grises y visualizarla utilizando OpenCV y Matplotlib.

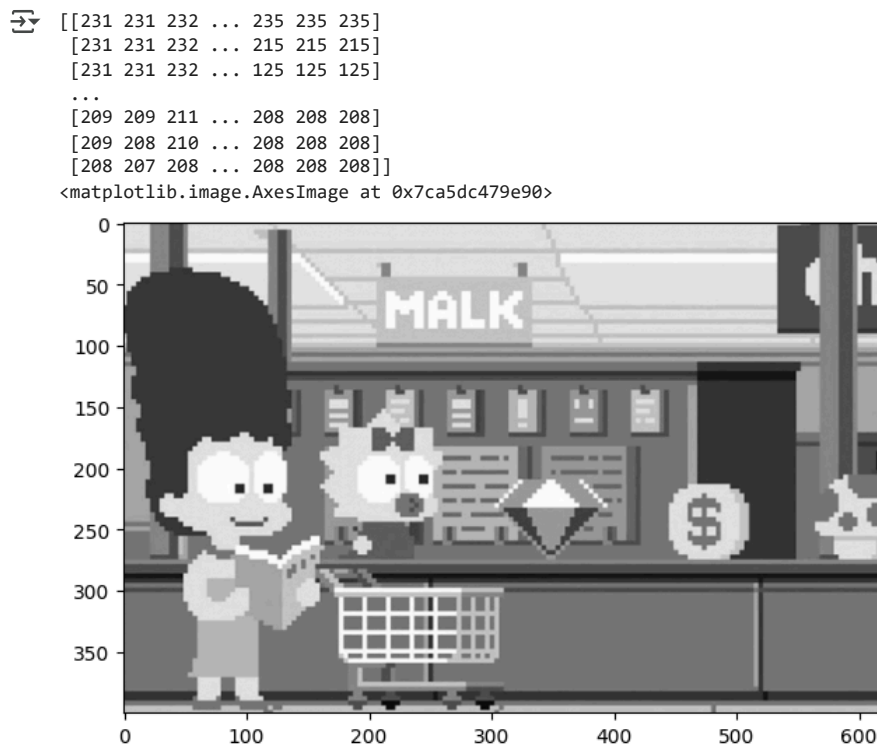
Pasos:

1. Cambia la variable `image_path` para que apunte al archivo de imagen de los simpson que esta en `data` . Asegúrate de que la ruta sea válida y que el archivo exista.
2. Usa la función `cv2.imread()` con el parámetro `cv2.IMREAD_GRAYSCALE` para cargar la imagen en escala de grises.
3. Muestra la imagen con `plt.imshow()` usando el mapa de colores `'gray'` .

```
image = cv2.imread("/content/drive/MyDrive/Colab Notebooks/1_Pixeles_Resolucion_Profundidad/data/simpson_grayscale.png", 0)
print(image)
```

```
# Modificamos los parámetros por defecto de matplotlib, todos los gráficos usarán estos parámetros
plt.rcParams['figure.figsize'] = (7.0, 7.0) # Tamaño de la figura
```

```
#Mostrar la imagen cargada
plt.imshow(image, cmap='gray')
```



```

from google.colab import drive
drive.mount('/content/drive')

```

Mounted at /content/drive

✓ Ejercicio 3: Análisis y manipulación de píxeles

Objetivo: Aprender a analizar las dimensiones de una imagen, acceder al valor de un píxel específico, modificarlo, y observar el resultado.

Pasos:

1. Determina las dimensiones de la imagen, Esto devolverá una tupla (alto, ancho) que indica las dimensiones de la imagen en píxeles.
2. Calcula el número total de píxeles, Usa `image.size` para saber cuántos píxeles contiene toda la imagen.
3. Accede a varios valor de un píxel en una posición específica (fila y columna).
4. Cambia los colores de los pixeles por otro.
5. Muestra la imagen con `plt.imshow()` usando el mapa de colores 'gray'.

```

#Dimensiones de la imagen
print(f"Dimensiones de la imagen: {image.shape} (Alto X Ancho)")
print(f"Numero total de pixeles: {image.size}")

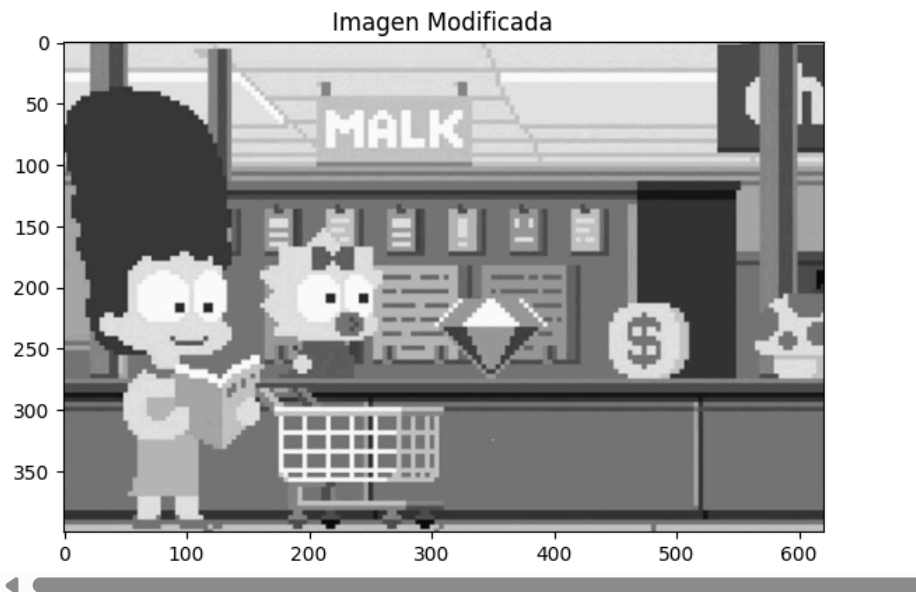
#Valor de un pixel en una posicion especifica
row, col = 325, 350
pixel_value = image[row, col]
print(f"Valor del pixel en la posicion ({row}, {col}): {pixel_value}")

#Mostrar el valor del pixel
image[row, col] = 255 #Cambiar a blanco
print(f"Nuevo valor del pixel en la posicion({row}, {col}): {image[row, col]}")

#Mostar la imagen modificada
plt.imshow(image, cmap='gray')
plt.title('Imagen Modificada')
plt.show()

```

↔ Dimensiones de la imagen: (400, 620) (Alto X Ancho)
Numero total de pixeles: 248000
Valor del pixel en la posicion (325, 350): 255
Nuevo valor del pixel en la posicion(325, 350): 255



✓ Ejercicio 4: Cambiar resolución y observar pixelación

Objetivo: Aprender a cambiar la resolución de una imagen (reducirla y luego escalarla de vuelta) para observar el efecto de la pixelación.

Pasos:

1. Usa la variable `scale_percent` para establecer cuánto quieres reducir el tamaño de la imagen.
2. Usa las dimensiones originales de la imagen (`image.shape`) y el porcentaje de escala para calcular el nuevo ancho y alto
3. Usa la función `cv2.resize()` con el método `cv2.INTER_AREA`, que es ideal para reducir imágenes sin perder demasiada calidad.
4. Muestra la imagen reducida con `plt.imshow()` para observar cómo se ve después de escalarla hacia abajo.
5. Usa `cv2.resize()` nuevamente para escalar la imagen reducida de vuelta a su tamaño original. En este caso, usa la interpolación `cv2.INTER_NEAREST`, que mantiene los bordes duros y crea un efecto de pixelación.
6. Muestra la imagen escalada con `plt.imshow()` de vuelta y nota cómo aparece un efecto de pixelación debido a la falta de detalles en la imagen reducida.

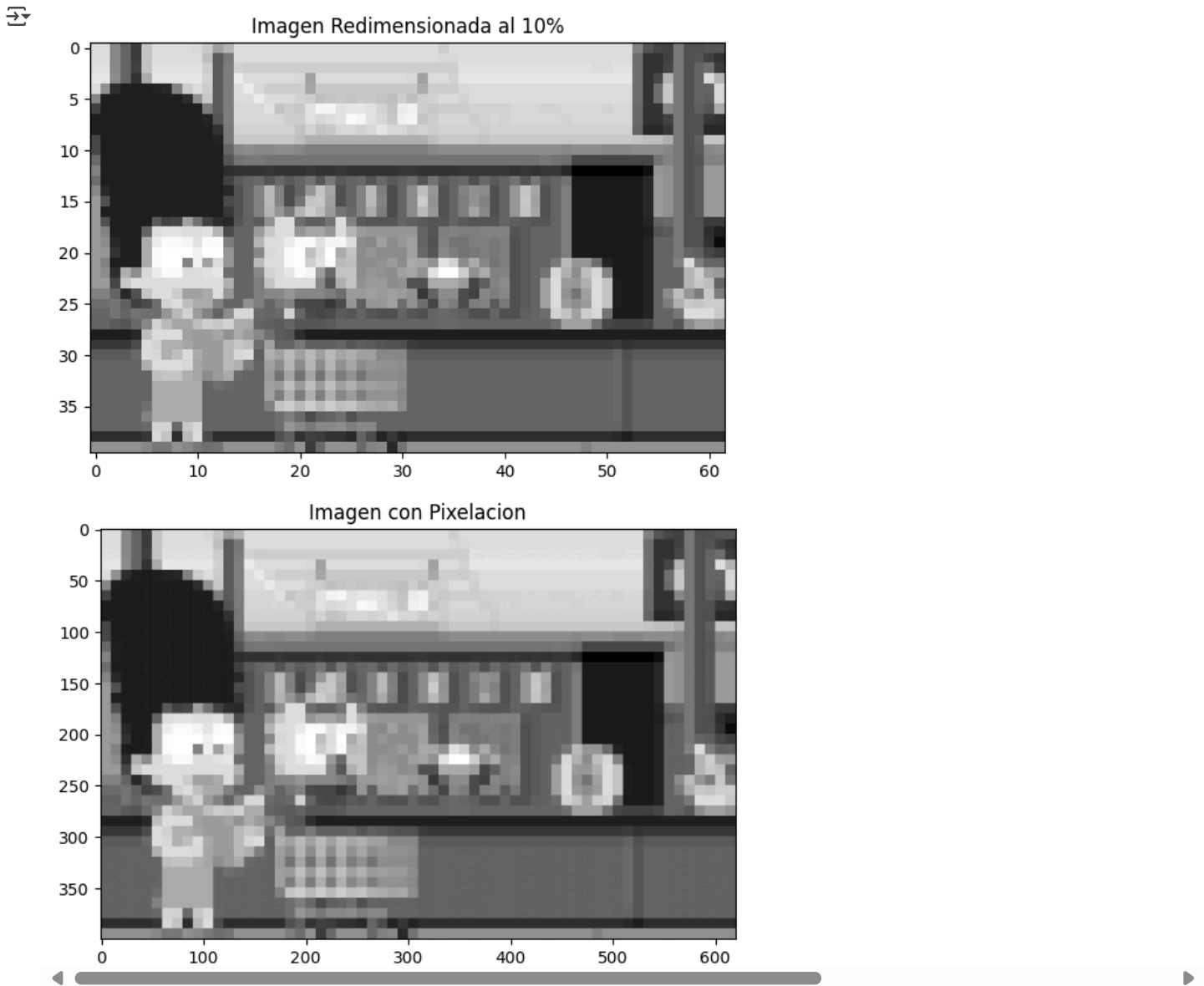
```
#Cambiar la resolucion de la imagen (escalado hacia abajo)
scale = 10 #Escalar al 10% del tamaño original
width = int(image.shape[1]*scale/100)
height = int(image.shape[0]*scale/100)
dim = (width, height)

#Redimensionar la imagen
resized_image = cv2.resize(image, dim, interpolation=cv2.INTER_AREA)

#Mostrar la imagen redimensionada
plt.imshow(resized_image, cmap='gray')
plt.title('Imagen Redimensionada al 10%')
plt.show()

#Volver a escalar al tamaño original para observar la pixelacion
upscaled_image = cv2.resize(resized_image, (image.shape[1], image.shape[0]), interpolation=cv2.INTER_NEAREST)

#Mostrar la imagen con pixelacion
plt.imshow(upscaled_image, cmap='gray')
plt.title('Imagen con Pixelacion')
plt.show()
```



✓ Ejercicio 5: Cambiar profundidad de bits

Objetivo: Reducir la profundidad de bits de una imagen en escala de grises, pasando de 8 bits (256 niveles de gris) a 2 bits (4 niveles de gris), y visualizar el efecto resultante.

Pasos:

1. Usa la operación matemática $(image // 64) * 64$ para agrupar los valores de los píxeles en 4 niveles de gris.
2. Usa `plt.imshow()` para mostrar la imagen resultante y observar cómo se pierden detalles debido a la reducción en los niveles de gris.

```
#Reducir la profundidad de bits de la imagen (8 bits a 2 bits)
reduced_bit_image = (image // 64) * 64
```

```
#Mostrar la imagen con profundidad de bits reducida
plt.imshow(reduced_bit_image, cmap='gray')
plt.title('Imagen con Profundidad de Bits Reducida (2 bits)')
plt.show()
```

